

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №3

Выполнил:
Афанасьев Николай
J3214

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Мигрировать ранее разработанное приложение LearnFlow (платформа для онлайн-курсов и обучения, вариант 1) с чистого HTML/CSS/JS на фреймворк Vue.JS.

Требования к приложению:

- подключён Vue Router с навигационными guard-ами;
- реализована работа с внешним API посредством библиотеки axios;
- разумное деление на компоненты (демонстрация компонентного подхода);
- использование composable-функций для выделения повторяющегося функционала.

Ход работы

Инициализация проекта

Проект создан с использованием сборщика Vite и шаблона Vue 3. В качестве зависимостей добавлены **vue-router** для маршрутизации, **axios** для работы с API и **json-server** для запуска мокового бэкенда.

Структура директорий:

```
1 src/  
2   assets/           --                               (CSS -  
3   composables/     -- useApi.js, useAuth.js, useCourses.js  
4   components/      -- AppSidebar.vue, CourseCard.vue, StatCard.vue,  
   LessonItem.vue  
5   views/           -- LoginView, RegisterView, DashboardView,  
   CoursesView,  
6   CourseView, SettingsView, TeacherView  
7   router/          -- index.js  
8   main.js  
9   App.vue
```

Роутер и навигационные гарды

Маршруты разделены на публичные (/login, /register) и защищённые (все остальные). В **router.beforeEach** реализована проверка сессии: незарегистрированный пользователь перенаправляется на страницу входа, а авторизованный не может открыть страницу логина.

```

1 router.beforeEach((to) => {
2   const { isLoggedIn, isTeacher } = useAuth()
3
4   if (to.meta.requiresAuth && !isLoggedIn.value) {
5     return { name: 'login' }
6   }
7   if (to.meta.authOnly && isLoggedIn.value) {
8     return { name: isTeacher.value ? 'teacher' : 'dashboard' }
9   }
10 })

```

Composables

Повторяющийся функционал вынесен в три composable-функции:

- **useApi** — обёртка над axios: методы `get`, `post`, `patch`, `del`. Единая точка работы с API, базовый URL задаётся один раз.
- **useAuth** — управление сессией через `localStorage`. Хранит реактивный объект `session`, вычисляемые свойства `isLoggedIn` и `isTeacher`, методы `setSession`, `clearSession`.
- **useCourses** — логика фильтрации каталога: реактивные поля поиска и фильтров, вычисляемый `filtered`, функция сброса. Используется в `CoursesView` и не дублируется в шаблоне.

Компоненты

- **AppSidebar** — боковая навигация. Динамически показывает пункты меню в зависимости от роли пользователя (студент / преподаватель). Содержит кнопку переключения светлой и тёмной темы (/), блок с именем пользователя и кнопку выхода.
- **CourseCard** — карточка курса со scored-стилями: тамбнейл с эмодзи, бейджи уровня и темы, рейтинг, цена. При клике переходит на страницу курса через `router.push`.
- **StatCard** — карточка статистики с тремя пропсами: `label`, `value`, `sub`. Используется на дашборде и в кабинете преподавателя.
- **LessonItem** — элемент списка уроков с состояниями: пройден, текущий, не начат.

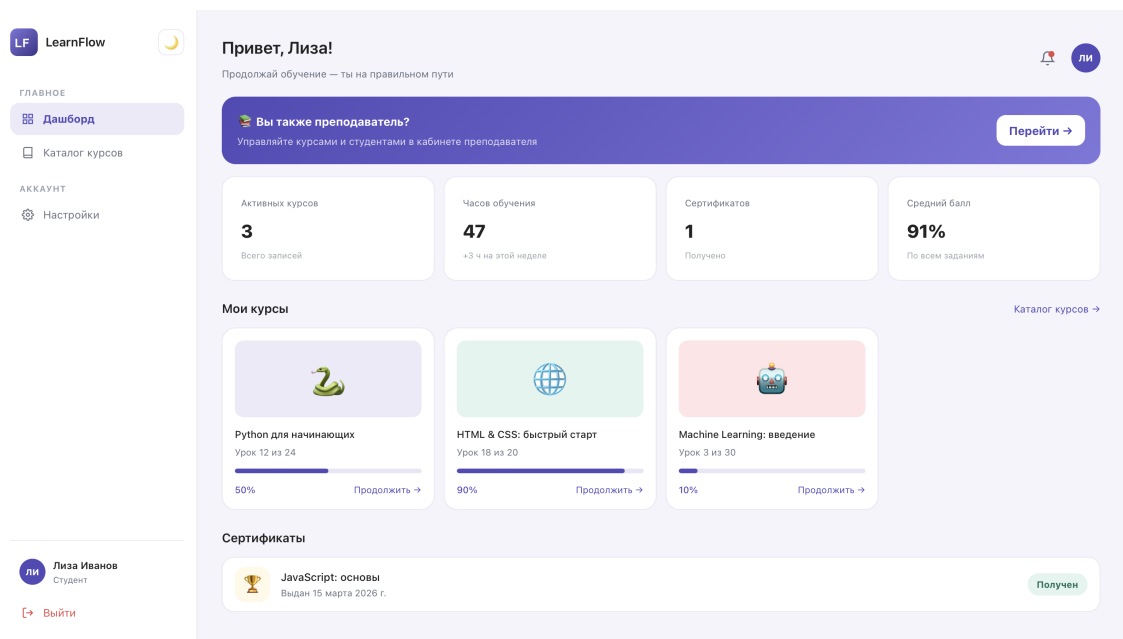


Рис. 1: Дашборд студента (светлая тема)

Страницы приложения

LoginView / RegisterView — формы авторизации и регистрации. Проверяют пользователя через API, сохраняют сессию, перенаправляют по роли.

DashboardView — личный кабинет студента. Загружает записи на курсы (`enrollments`) и сертификаты через `Promise.all`, отображает прогресс по каждому курсу.

CoursesView — каталог курсов с фильтрацией по названию, предмету, уровню и цене. Логика фильтрации реализована через `composable useCourses`, вся реактивность работает без перезапроса к серверу.

CourseView — страница курса. Если пользователь не записан, отображает кнопку «Записаться на курс». После записи появляется видеоплеер и список уроков с прогрессом.

SettingsView — настройки профиля. Позволяет редактировать имя, фамилию, email и «о себе». Содержит переключатель тёмной темы через `inject('toggleTheme')`.

TeacherView — кабинет преподавателя. Список курсов с возможностью публикации и удаления. Создание нового курса через модальное окно (`<Teleport to="body">`).

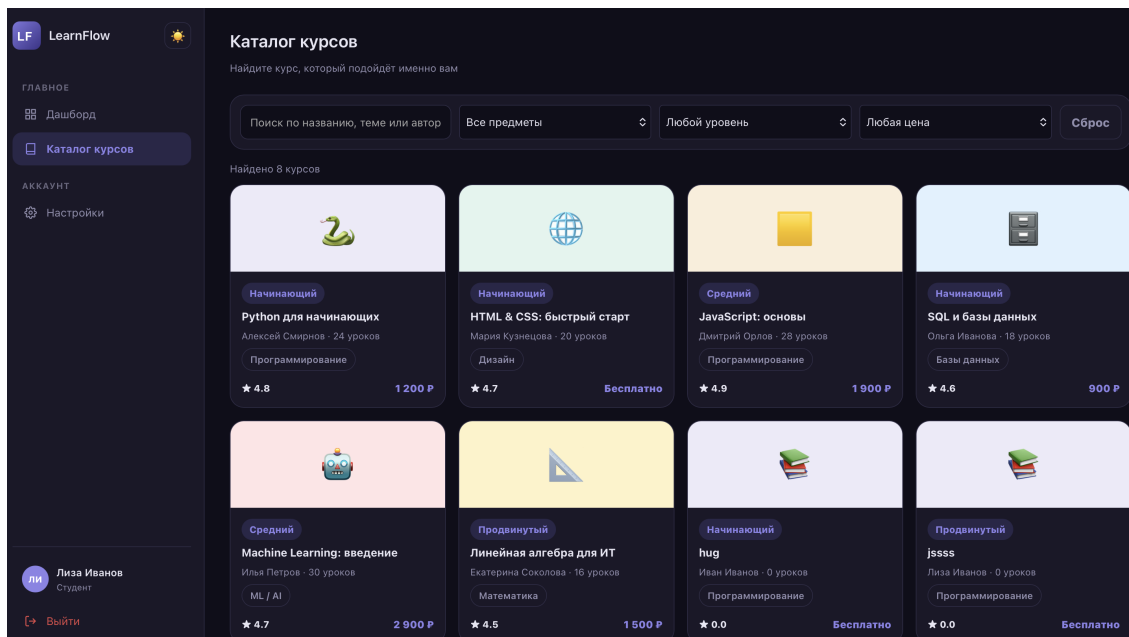


Рис. 2: Каталог курсов с фильтрацией (тёмная тема)

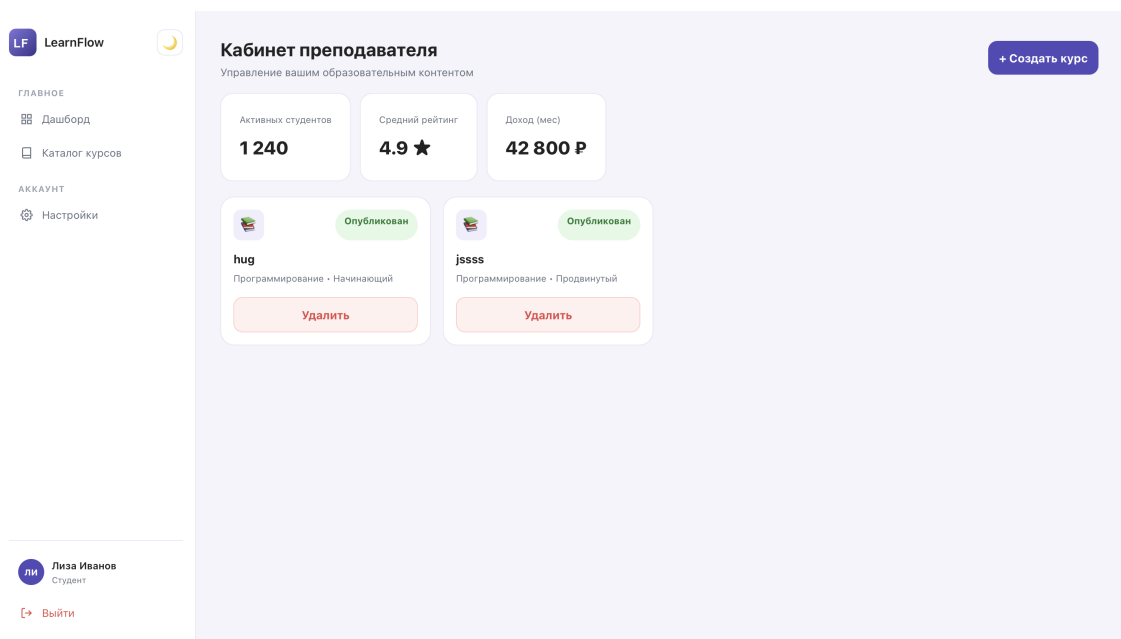


Рис. 3: Кабинет преподавателя

Темизация

Светлая и тёмная темы реализованы через CSS-переменные. Переключение происходит установкой атрибута `data-theme` на элемент `<html>`. Значение темы хранится в `localStorage` и восстанавливается при перезагрузке.

Запуск проекта

Проект запускается двумя командами в отдельных терминалах:

```
1 npm run api      # json-server      http://localhost:3000
2 npm run dev      # Vite dev-        http://localhost:5173
```

Вывод

В ходе лабораторной работы приложение LearnFlow было полностью перенесено с нативного HTML/CSS/JS на фреймворк Vue3. Реализована клиентская маршрутизация с защитой маршрутов, работа с моковым API через axios, компонентный подход с переиспользуемыми блоками и выделение повторяющейся логики в composable-функции. Приложение поддерживает две темы оформления и сохраняет сессию пользователя между перезагрузками.